Entrega 3 · Documento 5

Pruebas manuales complementarias

Análisis estático, escaneo activo, endurecimiento de contenedores y verificación del control anti-DoS

Sistema integrado de eventos laborales (JML)

Andina Servicios S.A.S. (empresa ficticia · datos ficticios)

Politécnico Grancolombiano · Gerencia de Proyectos en Seguridad de la Información · Grupo D01

24 de octubre de 2026

Todas las pruebas se ejecutaron contra el despliegue local del sistema (BunkerWeb + aplicación + PostgreSQL + GLPI 11 + MariaDB en Docker Compose). No se ejecutaron pruebas activas contra servicios de terceros (Supabase). Los secretos y tokens de las evidencias están enmascarados.

1. Resumen

Este documento complementa las pruebas automatizadas y los escáneres de los Documentos 1 a 3 con cinco técnicas manuales ejecutadas a mano contra el despliegue local: análisis estático del código (bandit), escaneo dinámico activo con inyección de cargas reales (OWASP ZAP), auditoría de endurecimiento de los contenedores (Docker Bench), inspección del tráfico capturado (Wireshark) y verificación del control de disponibilidad del WAF (límite de peticiones y baneo). Juntas cubren las tres perspectivas: caja blanca (código), caja negra (ataque desde fuera) y configuración (contenedores y host).

Técnica	Tipo	Resultado	Requisito
bandit	Estático (caja blanca)	0 hallazgos en 8509 líneas	RS-05, RS-11
OWASP ZAP activo	Dinámico con cargas reales	0 FAIL · 110 reglas PASS	RS-13, T4
Docker Bench	Config. de contenedores	105 chequeos; aislamiento OK	RS-14
Wireshark	Inspección de tráfico	TLS externo; restringido no sale a GLPI	RS-01, T7
Verificación anti-DoS	Control de disponibilidad	429 y baneo responden	RS-08, T13/T21

2. Alcance y advertencia

Todo se ejecutó contra el stack local (Docker Compose), sobre el propio sistema, como laboratorio autorizado. **No se escaneó Supabase** (servicio de terceros). El escaneo activo de ZAP y la verificación anti-DoS envían peticiones de ataque, por lo que se dirigieron al entorno local desechable, nunca a datos que importe conservar. La frontera entre verificar un control y ejecutar un ataque destructivo se respeta explícitamente en la sección 6. Evidencia cruda: `evidencias/vulnerabilidades/` (`bandit.html`, `zap-activo-sin-waf.*`, `docker-bench-completo.txt`) y `evidencias/trafico/` (capturas .pcap). Guía reproducible: `docs/probar-seguridad.md`.

3. Análisis estático del código (bandit)

bandit recorre el código Python en busca de patrones inseguros (SQL con f-strings, `shell=True`, deserialización peligrosa, secretos embebidos). Es la cara de caja blanca que complementa a ZAP y sqlmap. Corre en la CI y a mano con el mismo comando.

bandit sobre src (igual que la CI)

```
$ uvx bandit==1.9.4 -r src -x '*/migrations/*'


Test results: No issues identified.
Code scanned: Total lines of code: 8509
Run metrics (by severity): Undefined 0 · Low 0 · Medium 0 · High 0
Total issues skipped (#nosec, justificados): 3
```

Los 3 avisos deshabilitados con `#nosec` son usos revisados, no hallazgos: `B108` en `settings.py` (una ruta de latido bajo `/tmp`, no un archivo temporal inseguro) y `B308/B703` en `graficos.py` (el único `mark_safe` permitido, para el SVG que genera el servidor a partir de números, con etiquetas escapadas). Un hallazgo nuevo de severidad media o alta hace fallar la CI. Evidencia: `bandit.html`.

4. Escaneo dinámico activo (OWASP ZAP)

El escaneo **activo** va más allá del baseline (pasivo) del Documento 2: inyecta cargas de ataque reales contra cada parámetro. Se ejecutó autenticado y **contra Django directamente** (red interna), no a través del WAF, para medir el código sin el borde —atravesar el WAF con un escaneo activo solo provoca el baneo, como se vio en el Documento 2—.

Resultado del escaneo activo	Valor
Reglas que fallaron (FAIL)	0
Reglas superadas (PASS)	110
Avisos (WARN)	3 (bajo riesgo, por probar sin WAF)

Las reglas de inyección con cargas reales pasaron todas. Una muestra representativa:

Familia de ataque probada (activa)	Regla ZAP
Inyección SQL (genérica y por tiempo: PostgreSQL, MySQL, Oracle, MsSQL)	40018–40027
Inyección de comandos del SO (directa y por tiempo)	90020, 90037
Inyección de plantilla del servidor (SSTI, incluida ciega)	90035, 90036
Recorrido de rutas (Path Traversal)	6
Ejecución remota de código (Shell Shock, CVE-2012-1823)	10048, 20018
Entidad externa XML (XXE) y Padding Oracle	90023, 90024

Los 3 WARN son los mismos de bajo riesgo del baseline y se explican por probar sin el WAF: `X-Content-Type-Options` ausente (lo añade BunkerWeb, RS-07), `ACAO:*` en el CSS estático, y «HTTP Only Site» (la prueba fue por HTTP directo; en producción todo es HTTPS). El borde los resuelve. Evidencia: `zap-activo-sin-waf.html` y `.json`.

5. Endurecimiento de los contenedores (Docker Bench)

Docker Bench audita el stack contra el CIS Docker Benchmark (105 chequeos). Da evidencia directa de RS-14: contenedores sin privilegios, aislamiento del host, imágenes por digest y límites. Lo relevante del stack:

Chequeo	Resultado	Lectura
5.3 Capacidades restringidas	PASS	cap_drop: ALL en el compose
5.4 Sin contenedores privilegiados	PASS	Ninguno es --privileged
5.5 Sin directorios sensibles del host montados	PASS	—
5.6 Sin SSH dentro de contenedores	PASS	—
5.15–5.21 Namespaces, seccomp	PASS	Aislamiento del host correcto
5.31 Socket de Docker no montado	PASS	Ningún contenedor lo monta
5.1 AppArmor	WARN	Limitación de Docker Desktop; activar en la VM Linux de Azure
5.7 Puertos privilegiados (80/443)	WARN	Por diseño: solo el WAF los publica
5.11 Límite de CPU	WARN	Hay mem_limit; falta cpus (endurece T13)
5.12 rootfs de solo lectura	WARN parcial	web y worker sí (read_only); bases y reportería no pueden
Ejecución como root (postgres, mariadb)	WARN	Sus imágenes arrancan root para el chown y bajan de usuario

Resumen: 15 PASS y 11 WARN en la sección de contenedores (5.x); los PASS cubren el aislamiento y la restricción de privilegios, y los WARN son limitaciones del entorno de desarrollo (Docker Desktop/Windows) o endurecimientos adicionales para Azure, no fallos de diseño. Evidencia: `docker-bench-completo.txt`.

6. Verificación del control anti-DoS

Un pentest de este sistema no busca tumbar el servicio, sino comprobar que sus protecciones de disponibilidad responden. El sistema las tiene explícitas (RS-08; amenazas T13 y T21; criterio CA-14 «el sexto intento en un minuto se bloquea»): límite general de 2 peticiones por segundo por IP, 5 por minuto en `/ingreso` y TOTP, y baneo de una hora por mal comportamiento. Una ráfaga **acotada** (una docena de peticiones) confirma el control sin causar caída.

Verificación acotada del límite de peticiones (RS-08, CA-14)

```
$ for i in $(seq 1 12); do
curl -sk -o /dev/null -w "%{http_code} " --resolve app.localhost:443:127.0.0.1 \
https://app.localhost/ingreso
done
200 200 200 302 429 429 429 429 429 429 429 429 # el límite de 5/min dispara 429

$ docker compose -f infra/compose.yaml exec bw-scheduler bwcli bans
10.20.40.1 Service: app.localhost Reason: bad behavior (baneo de 1 h)

$ docker compose -f infra/compose.yaml exec bw-scheduler bwcli unban 10.20.40.1
```

Que aparezca el 429 y que la IP quede baneada **es** la evidencia de que el control funciona. Durante el pentest del Documento 2, nmap, ZAP, sqlmap y Nikto dispararon este mismo baneo de forma espontánea.

Frontera. Lo defensivo es esta verificación acotada del límite y el baneo. Queda fuera un ataque volumétrico real (flooding, agotamiento de recursos, amplificación) cuyo objetivo sea dejar el servicio caído: aun contra la propia máquina no aporta evidencia

que la prueba acotada no dé ya, y el objetivo del laboratorio —probar que el límite de peticiones y el baneo responden— se cumple sin provocar la caída.

7. Inspección del tráfico (Wireshark)

Wireshark es la cara gráfica del análisis con tshark del Documento 3. No hace falta volver a capturar: los `.pcap` ya están en `evidencias/trafico/` y se abren con filtros de visualización. Es observación pasiva del propio tráfico. Tres experimentos confirman las propiedades clave:

Experimento	Filtro / acción	Qué se ve
Confidencialidad externa	Follow → TLS Stream en bunkerweb.pcap	Contenido ilegible (cifrado, RS-01)
El restringido no sale a GLPI	frame contains "CENTINELA" en glpi.pcap	Cero paquetes (invariante 5, T7)
…pero sí se almacena	mismo filtro en postgres.pcap	Aparece: vive solo en PostgreSQL
Consultas parametrizadas	pgsql.query en postgres.pcap	Identificadores citados, sin concatenar input (T4)

El detalle y los recuentos están en el Documento 3. Aquí se recogen como procedimiento manual reproducible con una herramienta gráfica.

8. Conclusión

Las cinco técnicas manuales confirman, desde ángulos distintos, los mismos resultados que las pruebas automatizadas: el código no tiene patrones inseguros (bandit) ni es inyectable con cargas reales (ZAP activo, 0 FAIL / 110 PASS); los contenedores corren aislados y sin privilegios (Docker Bench); el tráfico externo viaja cifrado y el campo restringido no sale a GLPI (Wireshark); y el control de disponibilidad del WAF responde con 429 y baneo (verificación anti-DoS). Ningún hallazgo nuevo de severidad alta. Los puntos abiertos son los ya recogidos en el Documento 2 (mantenimiento de imágenes, endurecimiento adicional) y los propios de la fase de despliegue en Azure (AppArmor, límites de CPU, Lynis sobre la VM).